RAG Assignment: Baseline vs. Re-Ranking

Project workflow with recursive constraint checks. Deadline: **19 October 2026**. Deliverables: **GitHub link + report**.

How to use this: every constraint has an ID (C1-C10). After each stage you run **every gate written so far**. If a later change breaks an earlier constraint you will see it immediately. Fix the **earliest failing stage**, then re-run everything after it.

```
GATES = []                      # each stage registers its checks here
def gate(fn): GATES.append(fn); return fn

def run_all_gates(upto=None):
    """Re-run every gate registered so far (cumulative / recursive check)."""
    for g in GATES[:upto]:
        g()                     # each gate uses assert with a clear message
    print(f"{len(GATES[:upto])} gates passed")

# Call run_all_gates() at the END of every notebook section.
```

Constraint register (paste into the first notebook cell)

ID	Constraint (from the assignment)
C1	One complex, long PDF (annual report, 10-K, long regulation, long paper). Long enough that Top-10 is a small fraction of the corpus.
C2	Both pipelines use the exact same chunk corpus.
C3	Dense bi-encoder, cosine / inner product, retrieve Top 10.
C4	Baseline passes the Top 3 directly to the LLM giving Answer 1.
C5	Re-ranking takes the same Top 10 and scores all 10 (query, chunk) pairs with a cross-encoder.
C6	Re-sort descending, keep Top 3, identical prompt structure, giving Answer 2.
C7	At least 2 failure queries where baseline degrades but re-ranking restores relevance.
C8	Document-specific failures: keyword trap, exception / corner case, negation / constraint, other (chunk buried at rank 4-10 promoted into Top 3).
C9	Report covers: architecture, rank shifts + Answer 1 vs 2, latency table, scaling question.
C10	Code in public/shared GitHub repo, ideally one well-commented .ipynb. Report concise, relevant, in your OWN words, not AI-generated.

Stage 1: Pick the document (C1)

Choose something long and dense: a 10-K or annual report (100+ pages), a long regulation (EU AI Act, GDPR, RBI/SEBI master circular) or a 40+ page research paper / survey. A bi-encoder and cross-encoder can only be told apart when many chunks compete. In a 20-chunk document, Top-10 is half the corpus and rank shifts mean nothing. Prefer text with exceptions, prohibitions, defined terms and synonyms, because you need them for Stage 7.

Gate (comment: re-run ALL gates after this stage, not just this one)

```
@gate
def g1_doc():
    assert n_pages >= 100 or n_words >= 50_000, "C1: document too short"
    assert n_chunks >= 300, "C1: too few chunks for meaningful Top-10 vs Top-3"
```

Must satisfy before moving on

[ ] Document is long and complex (C1).

[ ] It contains exceptions, 'shall not' language and repeated concepts under different names.

Stage 2: Environment and models

Pin everything so the notebook runs top to bottom.

- **Bi-encoder:** BAAI/bge-small-en-v1.5, all-MiniLM-L6-v2, or Gemini embeddings.
- **Cross-encoder:** cross-encoder/ms-marco-MiniLM-L-6-v2 (the assignment writes 'msmarco', the real Hugging Face ID has a hyphen: ms-marco).
- **LLM:** any API or local model, temperature=0 for both pipelines so answers are comparable.
- **Vector store:** FAISS, Chroma, or plain NumPy (fine for a few thousand chunks).
- Set random seeds and write a requirements.txt. Load API keys from environment variables.

Must satisfy before moving on

[ ] One LLM config object reused by both pipelines (same model, same temperature).

[ ] No API key typed into the notebook (C10).

Stage 3: Ingestion and chunking (C2)

Extract text page by page (pypdf or pymupdf), strip headers/footers, split into ~300-500 token chunks with 10-15% overlap. Store chunk_id, page and text. **Build the chunk list once** and let both pipelines read that single variable.

Gate (comment: re-run ALL gates after this stage, not just this one)

```
@gate
def g2_chunks():
    assert all(c["text"].strip() for c in chunks), "empty chunk found"
    assert len({c["id"] for c in chunks}) == len(chunks), "duplicate chunk ids"
    assert 100 <= avg_tokens <= 600, "chunk size out of range"
```

Must satisfy before moving on

[ ] Single shared `chunks` variable (C2).

[ ] Page number kept per chunk, you need it for citing evidence in the report.

Stage 4: Build the index (C3)

Embed all chunks with the bi-encoder, normalise vectors, store them. Use the same model for query embedding.

Gate (comment: re-run ALL gates after this stage, not just this one)

```
@gate
def g4_index():
    assert index.ntotal == len(chunks), "C2: index and chunk list out of sync"
    assert abs(np.linalg.norm(emb[0]) - 1) < 1e-3, "embeddings not normalised (cosine != IP)"
```

Stage 5: Pipeline 1, baseline (C3, C4)

Flow: query -> embed -> Top 10 (cosine) -> take [:3] -> prompt -> Answer 1. Write retrieve_top10(query) and baseline(query). Time retrieval and generation separately. Keep the prompt in one PROMPT_TEMPLATE constant.

Gate (comment: re-run ALL gates after this stage, not just this one)

```
@gate
def g5_baseline():
    r = retrieve_top10("test query")
    assert len(r) == 10, "C3: must retrieve 10"
    assert [x.score for x in r] == sorted([x.score for x in r], reverse=True)
    assert len(baseline_context("test query")) == 3, "C4: Top 3 only"
```

Must satisfy before moving on

[ ] Exactly 10 retrieved, exactly 3 sent to the LLM (C3, C4).

Stage 6: Pipeline 2, re-ranking (C5, C6)

Flow: the SAME top10 -> 10 (query, chunk) pairs -> cross-encoder scores -> sort descending -> [:3] -> same PROMPT_TEMPLATE -> Answer 2. Do not re-retrieve. Save each candidate's bi_rank and ce_rank so rank shifts are computed automatically.

Gate (comment: re-run ALL gates after this stage, not just this one)

```
@gate
def g6_rerank():
    cand = retrieve_top10(q); rr = rerank(q, cand)
    assert {c.id for c in rr.all10} == {c.id for c in cand}, "C5: candidate set changed"
    assert len(rr.all10) == 10, "C5: all 10 pairs must be scored"
    assert len(rr.top3) == 3
    assert [c.ce_score for c in rr.all10] == sorted([c.ce_score for c in rr.all10], reverse=True)
    assert PROMPT_TEMPLATE_P1 is PROMPT_TEMPLATE_P2, "C6: prompt must be identical"
    assert llm_cfg_p1 == llm_cfg_p2, "same model and temperature in both pipelines"
```

Must satisfy before moving on

[ ] Same candidate set, same prompt, same LLM settings: the ONLY difference is the ordering (C5, C6).

Stage 7: Failure-mode hunting (C7, C8)

This takes the longest. Use a structured search:

- **Mine the document** for 'except', 'unless', 'shall not', 'prohibited', 'provided that', defined terms, and ideas with several names. Draft 3-5 candidate queries per failure type.
- **Hand-label a gold chunk** per query (the chunk that truly contains the answer). Record chunk_id and page.
- **Run both pipelines** and log gold_bi_rank and gold_ce_rank.
- **Keep a query as a failure case** only if the gold chunk is outside the baseline Top 3 (ideally ranks 4-10) AND inside the re-ranked Top 3. Otherwise discard it or keep it as an honest counter-example.
- **Cover every category:** keyword trap (high lexical overlap with an irrelevant passage), exception / corner case, negation / constraint, and one other buried-chunk case.
- Aim for 5-6 candidates so at least 2 qualifying wins survive. Read the Top-3 chunks and both answers yourself, and note whether Answer 1 hallucinated, omitted a clause, or misstated a rule.

Gate (comment: re-run ALL gates after this stage, not just this one)

```
@gate
def g7_failures():
    wins = [f for f in failures if f.gold_bi_rank > 3 and f.gold_ce_rank <= 3]
    assert len(wins) >= 2, "C7: need >=2 true rank-promotion cases"
    cats = {f.category for f in wins}
    assert {"keyword_trap","exception","negation"} <= cats, "C8: missing a failure category"
    assert all(f.gold_chunk_id for f in failures), "every query needs a hand-labelled gold chunk"
```

Must satisfy before moving on

[ ] At least 2 true rank-promotion cases (C7).

[ ] Keyword trap, exception and negation each represented (C8).

[ ] A table of query | category | gold chunk | bi-rank | ce-rank | Answer 1 verdict | Answer 2 verdict, saved for the report.

Stage 8: Latency profiling (C9, item 3)

- Warm-up run first so model loading does not skew results.
- Run each pipeline on the same queries at least 5-10 times; report mean +/- std.
- Time separately: query embedding + vector search, cross-encoder scoring (10 pairs), LLM generation.
- Tabulate baseline retrieval vs retrieval + re-rank (optionally end-to-end with LLM). State whether you used CPU or GPU.

Gate (comment: re-run ALL gates after this stage, not just this one)

```
@gate
def g8_latency():
    assert latency_df.notna().all().all(), "missing timings"
    assert (latency_df["rerank_total"] >= latency_df["baseline_retrieval"]).all(), \
        "re-rank pipeline includes baseline retrieval, so it must be >="
```

Must satisfy before moving on

[ ] Latency table exists with mean +/- std and hardware noted (C9).

Stage 9: Write the report (C9, C10)

Comment: the report must be written by you, in your own words, concise and directly relevant. It is evaluated on your understanding, not polish. Do not paste AI-generated text. Use this as a checklist of what to cover, not as text to copy.

Section	What it must contain
1. Architecture comparison	Bi-encoder: query and chunk are embedded independently, compared by cosine similarity, so chunk vectors can be precomputed. Cross-encoder: query and chunk are fed together and self-attention runs across all their tokens jointly, giving a relevance score. Mention what each costs and gains.
2. Qualitative and rank-shift analysis	For each failure query: a line like 'Chunk 212 moved Rank #8 -> #1'. Then Answer 1 vs Answer 2: did the baseline hallucinate, omit an essential clause, or misstate a rule because the key chunk sat outside the Top 3? Name the specific error type.
3. Latency profiling	Your table (baseline retrieval vs re-ranking pipeline) plus 1-2 sentences interpreting the overhead.
4. Scaling question	Cross-encoder needs one forward pass per (query, chunk) pair and nothing can be precomputed, so cost grows with corpus size N for every query. Bi-encoder embeds chunks once offline and searches by vector similarity (can use approximate nearest neighbour). Hence: cheap retrieval first, expensive re-ranking only on the small candidate set.

Report must satisfy before submission

[ ] All four sections above are present (C9).

[ ] Written in my own words, no pasted AI text (C10).

[ ] Concise: every paragraph answers something the assignment asked.

[ ] Rank-shift table included for every failure query, with page numbers of gold chunks.

[ ] At least one honest counter-example or limitation mentioned (shows real understanding).

[ ] Document named, with page count and chunk count stated.

[ ] Models used (bi-encoder, cross-encoder, LLM) and CPU/GPU stated.

Stage 10: Final gate and submission (C10)

```
@gate
def g10_submission():
    assert os.path.exists("RAG_Assignment.ipynb")
    assert notebook_runs_top_to_bottom(), "Restart kernel -> Run All must succeed"
    assert no_api_keys_in_repo(), "load keys from env vars; do not commit secrets"
    assert report_file_exists and github_repo_is_public_or_shared
```

Finish with **Restart kernel -> Run all -> run_all_gates()**. If any gate fails, go back to that stage, fix it, and re-run everything after it.

Final submission checklist (everything to satisfy at the end)

Tick every box before 19 October 2026

[ ] C1: Long, complex PDF chosen (100+ pages or 50k+ words).

[ ] C2: Both pipelines read the exact same chunk list.

[ ] C3: Bi-encoder, cosine / inner product, Top 10 retrieved.

[ ] C4: Baseline sends Top 3 to the LLM, Answer 1 saved.

[ ] C5: Same Top 10, all 10 pairs scored by the cross-encoder.

[ ] C6: Re-sorted, Top 3, identical prompt, Answer 2 saved.

[ ] C7: At least 2 queries where baseline fails and re-ranking fixes it.

[ ] C8: Keyword trap, exception / corner case, negation, plus one buried-chunk case shown.

[ ] C9: Report has architecture, rank shifts + Answer 1 vs 2, latency table, scaling answer.

[ ] C10: Single well-commented .ipynb in a public/shared GitHub repo, runs top to bottom, no API keys committed.

[ ] C10: Report is in my own words, concise, not AI-generated.

[ ] GitHub link and report both submitted.

[ ] run_all_gates() prints all gates passed after a fresh Run All.